

Design Tic-Tac-Toe

...ఆపై $N \times N$, ఆపై Connect-4

LeetCode 348 · Amazon · Microsoft · Adobe – ఇది interview lo ఎప్పుడూ ఒక్క ఆట కాదు. మొదట 3×3 , ఆపై " $N \times N$ చెయ్యి", ఆపై "Connect-4 చెయ్యి".

మూడు విరుపులు. మొదటిది ఒక ఎత్తుకి 1.8 లక్షల గడులు తనిఖీ చేస్తుంది. రెండోది Gomoku lo గెలుపుని మిస్ చేస్తుంది – ఆపై గెలవని వారిని గెలిపిస్తుంది. మూడోది ఒకరి ఎత్తుని తుడిచేస్తుంది.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడే ప్రతి output, ప్రతి సంఖ్య నిజంగా `node` `io run` చేసినదే.

ఈ **PROBLEM** గురించి నిజం – మరియు దాన్ని ముందే చెప్పాను

Tic-tac-toe ఒక్కటే ఒక interview problem కాదు. 3×3 board, 9 గడులు – ఏ పద్ధతైనా పనిచేస్తుంది. దాన్ని 20 నిమిషాల్లో రాసేసి కూర్చుంటే, మిగిలిన 25 నిమిషాలూ ఏమి చేస్తారు?

అందుకే interviewer **ఎప్పుడూ దాన్ని పించుతాడు:** "N×N చెయ్యి", "వరుసగా K ఉంటే గెలుపు అను", "Connect-4 గా మార్చు". మరియు అక్కడే ప్రతి సులభమైన పరిష్కారమూ విరుగుతుంది.

ఈ doc ఆ దారినే నడుస్తుంది – 3×3 నుంచి మొదలుపెట్టి, ప్రతి అడుగులోనూ విరగొట్టి, చివరికి **ఒకే code మూడు ఆటలు ఆడేలా** చేస్తుంది.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు అది ఎక్కడికి వెళుతుంది
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: ప్రతి ఎత్తుకీ మొత్తం board

3. Step – ప్రతి ఎత్తు తర్వాత అన్ని రేఖలనీ తనిఖీ
4. మొదటి విరుపు – ఒక ఎత్తుకీ 1,80,600 గడులు
5. Step – ప్రతి రేఖకీ ఒక గణన

Part 3 – రెండో విరుపు: K వరుసగా

6. Step – "ఇప్పుడు Gomoku చెయ్యి"
7. రెండో విరుపు – గెలుపు కనిపించలేదు · ఆపై గెలవనివాడు గెలిచాడు
8. Step – చివరి ఎత్తు గుండా వరుసని లెక్కించడం

Part 4 – మూడో విరుపు: ఆట స్థితి లేదు

9. Step – నియమాలు ఎక్కడ?
10. మూడో విరుపు – ఒకరి ఎత్తు తుడిచిపెట్టుకుపోయింది
11. Step – `Game` ఒక స్థితి యంత్రం

Part 5 – ఒక engine, మూడు ఆటలు

12. Step – ఎత్తు నియమాన్ని బయటికి తీయడం · గురుత్వాకర్షణ
13. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

Part 6 – Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి
15. నోటీతో చెప్పాల్సిన English script
16. Follow-ups – AI, undo, ఆన్లైన్ ఆట
17. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు అది ఎక్కడికి వెళుతుంది

"Design a tic-tac-toe game."

ఇది ప్రశ్న కాదు – ఇది ప్రారంభ బిందువు.

INTERVIEW యొక్క నిజమైన ఆకారం

ప్రతి అడుగులోనూ ముందటి పరిష్కారం విరుగుతుంది



మరియు ఒక నాలుగో విషయం – ఎవరూ అడగనిది

"ఆట ముగిసింది" అంటే ఏమిటి? ఎవరి వంతు? ఆక్రమించిన గడి మీద ఆడితే?

ఇవి win check కోసం ముందు ఉండాల్సిన నియమాలు – మరియు చాలా మంది వీటిని అస్సలు రాయరు (§10).

ఈ doc చివరికి – ఒక engine మూడు ఆటలూ అడుతుంది.

Tic-tac-toe అంటే

`KInARow(3)` + `PlaceAnywhere` .Connect-4 అంటే `KInARow(4)` + `DropInColumn` . Game class lo ఒక్క మార్పు లేదు.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
Board పరిమాణం స్థిరమా, $N \times N$ నా?	$N \times N$ అంటే §4 – win check ఖర్చు ముఖ్యమవుతుంది
గెలవడానికి పూర్తి వరుస కావాలా, K వరుసగా చాలా?	$K < N$ అంటే పూర్తిగా వేరే algorithm (§8)

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
ఇద్దరే ఆటగాళ్ళు, ఎక్కువమందా?	ఎక్కువమంది అయితే turn logic సాధారణీకరించాలి
గురుత్వాకర్షణ ఉందా (Connect-4)?	అవును అంటే ఎత్తు నియమం వేరే (§12)
సమం (draw) ఎలా నిర్ణయించాలి?	§11 – మరియు చాలా మంది దీన్ని మర్చిపోతారు
Undo కావాలా?	అవును అంటే ఎత్తుల చరిత్ర (§16)
AI ఎదురుగా ఆడాలా?	§16 – ఇది ఒక వేరే layer

రెండో ప్రశ్న అతి ముఖ్యమైనది – మరియు దాదాపు ఎవరూ అడగరు

"గెలవడానికి మొత్తం వరుస కావాలా, లేక వరుసగా K ఉంటే చాలా?"

Tic-tac-toe lo ఇవి ఒకటే – 3×3 board, 3 వరుసగా = పూర్తి వరుస. కాబట్టి తేడా కనిపించదు.

కానీ Gomoku lo board 15×15 , గెలుపు 5 వరుసగా. ఇప్పుడు ఆ రెండూ పూర్తిగా వేర్వేరు problems – మరియు tic-tac-toe కి పనిచేసే అందమైన పరిష్కారం (§5) ఇక్కడ నిశ్శబ్దంగా తప్పు జవాబులు ఇస్తుంది (§7).

ఈ ప్రశ్న అడగడం అంటే – మీరు ముందు చూస్తున్నారని చూపించడం.

Part 2 – మొదటి విరుపు: ప్రతి ఎత్తుకీ మొత్తం board

3. Step – ప్రతి ఎత్తు తర్వాత అన్ని రేఖలనీ తనిఖీ

```
class Game {
  constructor(n = 3) {
    this.n = n;
    this.board = Array.from({ length: n }, () => Array(n).fill(null));
    this.turn = 'X';
  }
  play(r, c) {
    this.board[r][c] = this.turn;
    const won = this.checkWin(this.turn);
    this.turn = this.turn === 'X' ? 'O' : 'X';
    return won ? `${this.board[r][c]} గెలిచాడు` : null;
  }
  checkWin(p) {
    const n = this.n, b = this.board;
    for (let i = 0; i < n; i++) { // ప్రతి వరుస
      if (b[i].every(x => x === p)) return true;
      let col = true;
      for (let j = 0; j < n; j++) if (b[j][i] !== p) { col = false; break; }
      if (col) return true;
    }
    let d1 = true, d2 = true; // రెండు వికర్ణాలు
    for (let i = 0; i < n; i++) {
      if (b[i][i] !== p) d1 = false;
      if (b[i][n-1-i] !== p) d2 = false;
    }
    return d1 || d2;
  }
}
```

పనిచేస్తుంది, మరియు చదవడానికి సృష్టం:

```
(0,0) → ...
(1,1) → ...
(0,1) → ...
(2,2) → ...
(0,2) → X గెలిచాడు

X X X
. O .
. . O
```

3×3 కి ఇది పరిపూర్ణం. Interviewer అంటాడు: "ఇప్పుడు $N \times N$ చెయ్యి."

4. మొదటి విరుపు – ఒక ఎత్తుకి 1,80,600 గడులు

ఒక పూర్తి ఆట (n^2 ఎత్తులు) ఆడి, ఎన్ని గడులు తనిఖీ అయ్యాయో లెక్కిద్దాం:

board	ఎత్తులు	గడి తనిఖీలు	ఒక్కో ఎత్తుకి	సమయం
3×3	9	216	24	0 ms
20×20	400	3,36,000	840	3 ms
100×100	10000	20,20,00,000	20,200	425 ms
300×300	90000	16,25,40,00,000	1,80,600	35275 ms

ఒక్కో ఎత్తు వేయడానికి చేసే పని

Board పరిగీతే, ఒక్కో ఎత్తు ఖరీదవుతుంది – ఎత్తు మాత్రం అదే ఒక్కో గడి

3×3 24 తనిఖీలు

20×20 840

100×100 20,200

300×300 1,80,600 తనిఖీలు – ఒక్కో రాయి పెట్టడానికి

ఆట మొత్తం: 16,25,40,00,000 గడి తనిఖీలు · 35 సెకన్లు

మొదటి విరుపు – మారినది ఒక గడి, తనిఖీ చేసింది మొత్తం BOARD

ఆటగాడు ఒక్కో గడిని మార్చాడు. System $2n+2$ రేఖలని – అంటే $\sim n^2$ గడులని – తనిఖీ చేసింది.

మరియు ఆలోచించండి: (0,0) వద్ద ఒక రాయి పెడితే, వరుస 250 మారే అవకాశమే లేదు. అయినా దాన్ని చూస్తున్నాం.

మౌలిక తప్పు: మనం "ఎవరైనా గెలిచారా?" అని అడుగుతున్నాం. సరైన ప్రశ్న: "ఈ ఎత్తు గెలిపించిందా?" – ఎందుకంటే ఆట మిగతా భాగం మారనేలేదు.

ఇది ఈ series lo పదేపదే వచ్చిన అదే వాక్యం: ఖర్చు మారిన దాని మీద ఆధారపడాలి (Deep Dive 12 §5, 13 §4, 14 §11).

5. Step – ప్రతి రేఖకీ ఒక గణన

Tic-tac-toe lo గెలుపు అంటే ఒక పూర్తి రేఖ. కాబట్టి ప్రతి రేఖకీ ఒక నడుస్తున్న గణన ఉంచితే చాలు – X కి +1, O కి -1 :

```
class CounterGame {
    constructor(n = 3) {
        this.n = n;
        this.rows = new Int32Array(n);
    }
}
```

```

    this.cols = new Int32Array(n);
    this.diag = 0; this.anti = 0;
}
// X → +1, 0 → -1. ఒక రేఖ మొత్తం |n| అయితే ఆ రేఖ పూర్తయినట్టు.
place(r, c, p) {
    const v = p === 'X' ? 1 : -1, n = this.n;
    this.rows[r] += v; this.cols[c] += v;
    if (r === c) this.diag += v;
    if (r + c === n - 1) this.anti += v;
    return Math.abs(this.rows[r]) === n || Math.abs(this.cols[c]) === n ||
           Math.abs(this.diag) === n || Math.abs(this.anti) === n;
}
}

```

ఆ **+1 / -1 ఒక చక్కని ఉపాయం**. ఒక రేఖలో X లు మరియు O లు కలిసి ఉంటే, గణన ఎప్పటికీ $\pm n$ కి చేరదు – ఎందుకంటే అవి ఒకదాన్నొకటి కొట్టేస్తాయి. కాబట్టి "ఈ రేఖలో అన్నీ ఒకే ఆటగాడివేనా?" అనే ప్రశ్నకి ఒక్క **పూర్ణాంకం** సరిపోతుంది – మొత్తం రేఖని చూడాల్సిన అవసరం లేదు.

ఇది **LeetCode 348** యొక్క కచ్చితమైన జవాబు, మరియు interview lo ఇది ఒక మంచి క్షణం.

board	ఎత్తులు	పని	ఒక్కో ఎత్తుకి	సమయం
3×3	9	36	4	8 ms
20×20	400	1,600	4	0 ms
100×100	10000	40,000	4	1 ms
300×300	90000	3,60,000	4	2 ms

నాలుగు operations – నాలుగు సందర్భాల్లోనూ. మరియు **35,275 ms → 2 ms**.

ఆ 3×3 వరుసలో **8 ms** చూడండి – 300×300 కి 2 ms కంటే ఎక్కువ. అది JIT వేడెక్కడం (మొదటి iteration), ఆటకీ సంబంధించినది కాదు. **ఇంత చిన్న సమయాలని నమ్మొద్దు** – పని నిలుపు వరుస (ఎప్పుడూ 4) మాత్రమే నిజమైన కౌలత.

Part 3 – రెండో విరుపు: K వరుసగా

6. Step – "ఇప్పుడు Gomoku చెయ్యి"

Interviewer అంటాడు: "Board 15×15 , మరియు వరుసగా 5 ఉంటే గెలుపు."

సహజమైన ప్రతిస్పందన: "సులభం – n కి బదులు 5 పెడతాను." మరి ముందు అది ఏమి చెప్తుందో చూద్దాం.

7. రెండో విరుపు – గెలుపు కనిపించలేదు • ఆపై గెలవనివాడు గెలిచాడు

మొదటి సగం – గెలుపు మిస్ అయింది

X వరుస 7 లో (7,3) నుంచి (7,7) వరకు – సరిగ్గా ఐదు, వరుసగా:

Gomoku – 15×15 board, వరుసగా 5 ఉంటే గెలుపు

X (7,3) → ...
X (7,4) → ...
X (7,5) → ...
X (7,6) → ...
X (7,7) → ...

వరుస 7: ...XXXXX.....

→ X కి వరుసగా 5 ఉన్నాయి. System ఏమీ చెప్పలేదు.
ఎందుకంటే `rows[7] = 5`, మరియు అది `|15|` కావాలని చూస్తోంది.

రెండో సగం – "సరిచేస్తే" ఇంకా ఘోరం

సహజమైన సవరణ: `=== n` కి బదులు `>= K`.

```
place(r, c, p) {  
  this.rows[r] += p === 'X' ? 1 : -1;  
  return Math.abs(this.rows[r]) >= K;           // ← "సరిచేసినది"  
}
```

"|గణన| >= 5 అయితే గెలుపు" అని సరిచేస్తే –

X (7,0) → ... X.....
X (7,2) → ... X.X.....
X (7,4) → ... X.X.X.....
X (7,6) → ... X.X.X.X.....
X (7,8) → x "X గెలిచాడు" X.X.X.X.X.....

→ ఏ రెండు X లూ పక్కపక్కన లేవు. అయినా "గెలిచాడు".

ఇంకో సందర్భం – X 8, 0 3, కానీ ఏవీ వరుసగా లేవు:

X (7,13) → x "X గెలిచాడు" X0X0XX0X.X.X.X.

→ X లు 5 వరుసగా లేవు. rows[7] = 5

రెండో విరుపు – గణన "ఎన్ని" చెప్తుంది, "ఎక్కడ" చెప్తుంది

మొదట అది నిజమైన గెలుపుని మిస్ చేసింది. ఆపై "సరిచేశాక" అది గెలవనివారిని గెలిపించింది – రెండు రకాలుగా:

1 • X.X.X.X.X – ఐదు X లు, ఏ రెండూ పక్కపక్కన లేవు.

2 • X0X0XX0X.X.X.X. – 8 X, 3 O; గణన 5, కానీ వరుసగా 2 మాత్రమే.

మౌలిక తప్పు: §5 యొక్క ఉపాయం ఒక దాచిన ఊహ మీద నిలబడింది – "గెలుపు అంటే పూర్తి రేఖ". అప్పుడు "ఎన్ని" మరియు "ఎక్కడ" ఒకటే: n ఉంటే అవి వరుసగా ఉండాలిందే.

K < n అయిన క్షణం ఆ ఊహ చనిపోతుంది, కానీ code అలాగే ఉంటుంది. అందుకే ఇది ఒక error ఇవ్వదు – అది నిశ్శబ్దంగా తప్పు జవాబులు ఇస్తుంది.

ఇదే నిజమైన interview ఉచ్చు. §5 యొక్క counter ఉపాయాన్ని చాలా మంది గుర్తుపెట్టుకుని వస్తారు (LeetCode 348). Interviewer "ఇప్పుడు K వరుసగా" అన్నప్పుడు వారు $\geq K$ అని మార్చి ముందుకీ వెళ్ళిపోతారు.

ఒక్క ఉదాహరణ గీస్తే చాలు – X.X.X.X.X – మరియు మీరు ఆ ఉచ్చులో పడలేదని స్పష్టమవుతుంది.

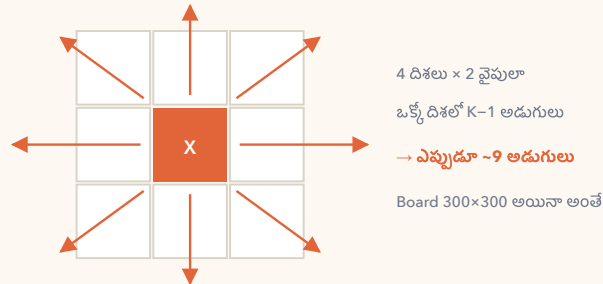
8. Step – చివరి ఎత్తు గుండా వరుసని లెక్కించడం

§4 lo అడిగిన ప్రశ్నకీ తిరిగి వద్దాం: "ఈ ఎత్తు గెలిపించిందా?"

ఒక రాయి (r,c) వద్ద పెట్టాక, గెలుపు వరుస ఆ రాయి గుండా వెళ్ళాలి. కాబట్టి నాలుగు దిశల్లో, రెండు వైపులా లెక్కిస్తే చాలు:

చివరి రాయి నుంచి నాలుగు దిశలు

కొత్త రాయి గుండా వెళ్ళని ఏ వరుసా ఈ ఎత్తుతో మారలేదు



8 దిశలు కాదు, 4 – ఎందుకంటే ప్రతి దిశనీ రెండు వైపులా ఒకేసారి లెక్కిస్తున్నాం.

```
const DIRS = [[0,1],[1,0],[1,1],[1,-1]]; // అడ్డం · నిలువు · రెండు వికర్ణాలు

class KInARow {
  constructor(k) { this.k = k; }
  // చివరి ఎత్తు (r,c) గుండా మాత్రమే చూస్తాం - అదే మారిన ఏకైక విషయం
  wins(board, r, c, p) {
    for (const [dr, dc] of DIRS) {
      let run = 1; // ఇప్పుడే పెట్టిన రాయి
      for (const sign of [1, -1]) { // రెండు వైపులా
        let rr = r + dr*sign, cc = c + dc*sign;
        while (board.at(rr, cc) === p) {
          if (++run >= this.k) return true;
          rr += dr*sign; cc += dc*sign;
        }
      }
      if (run >= this.k) return true;
    }
    return false;
  }
}
```

ముందు: ఇది నిజంగా సరైనదేనా?

ఇక్కడ చాలా సరిహద్దులు ఉన్నాయి – board అంచులు, రెండు వైపులా కలిపే వరుసలు, $K = N$ సందర్భం. కాబట్టి **fuzz test**: యాదృచ్ఛిక ఆటలు ఆడి, ప్రతి ఎత్తులోనూ ఒక **brute-force scan** తో పోల్చడం.

```
// brute force: మొత్తం board ని scan చేసి k వరుసలు వెతకడం
function bruteWins(board, n, k, p) {
  for (let r = 0; r < n; r++)
    for (let c = 0; c < n; c++)
      for (const [dr, dc] of DIRS) {
        let ok = true;
        for (let i = 0; i < k; i++) {
          const rr = r + dr*i, cc = c + dc*i;
          if (rr < 0 || rr >= n || cc < 0 || cc >= n || board[rr][cc] !== p)
            { ok = false; break; }
        }
        if (ok) return true;
      }
  return false;
}
```

720 ఆటలు · 32,287 ఎత్తులు · 6 వేర్వేరు (n,k) జతలు
run-checker = brute force? అవును ✓

ఇప్పుడు వేగం:

ఒక పూర్తి board (n^2 ఎత్తులు) · $k = 5$

board	ఎత్తులు	brute scan	run check	ఒక్కో ఎత్తుకి
10×10	100	3 ms	0 ms	10.4 అడుగులు
50×50	2500	13 ms	1 ms	9.4 అడుగులు

150×150	22500	237 ms	1 ms	9.1 అడుగులు
300×300	90000	1911 ms	7 ms	9.1 అడుగులు

Brute scan వరుస runs మధ్య ఉగుతుంది – 1-3 ms, 8-13 ms, 235-237 ms, 1,885-1,911 ms. Run check వరుస 0-7 ms. ఇక్కడా ఒక్కోసంఖ్యని కాదు, **రెండు వరుసల ఆకారాన్ని** చూడండి: ఒకటి board తో పాటు పెరుగుతోంది, రెండోది పెరగట్లేదు. "అడుగులు" నిలుపు వరుస మాత్రమే స్థిరమైన కొలత.

~9 అడుగులు, board పరిమాణంతో సంబంధం లేకుండా. మరియు ఇది **$K = N$ కి కూడా సరైనది** – tic-tac-toe కి కూడా ఇదే వాడొచ్చు.

ఒక పరిష్కారం, రెండు ఆటలు – మరియు అదే ముఖ్యం

§5 యొక్క counters **వేగవంతమైనవి** (4 అడుగులు vs 9), కానీ అవి **ఒక్క ఆటకే** పనిచేస్తాయి.

§8 యొక్క run check కొంచెం ఖరీదు, కానీ అది **tic-tac-toe, Gomoku, Connect-4 – అన్నిటికీ** పనిచేస్తుంది.

Interview lo ఈ ఎంపికని బయటపెట్టి చెప్పండి: "The counter trick is $O(1)$ but assumes k equals n . I'd use directional run counting instead – it's $O(k)$ but it generalises, and k is small." ఇది మీరు **ఒక trade-off ని తెలిసి తీసుకుంటున్నారని** చూపిస్తుంది, ఒక ఉపాయాన్ని గుర్తుపెట్టుకున్నారని కాదు.

Part 4 – మూడో విరుపు: ఆట స్థితి లేదు

9. Step – నియమాలు ఎక్కడ?

§3 యొక్క `play` ని మళ్ళీ చూడండి:

```
play(r, c) {  
  this.board[r][c] = this.turn;           // ← ఎలాంటి తనిఖీ లేదు  
  const won = this.checkWin(this.turn);  
  this.turn = this.turn === 'X' ? 'O' : 'X';  
  return won ? `${this.board[r][c]} గెలిచాడు` : null;  
}
```

Win check మీద మనం చాలా కష్టపడ్డాం. కానీ దానికి **ముందు** ఏమి ఉండాలి? – ఆ గడి ఖాళీయేనా? ఆట ఇంకా నడుస్తోందా? ఎవరి వంతు?

ఒక్కటే లేదు. నడిపి చూద్దాం.

10. మూడో విరుపు – ఒకరి ఎత్తు తుడిచిపెట్టుకుపోయింది

- ఆక్రమించిన గడి మీద ఆడితే
X (0,0) తర్వాత:
X . .
. . .
. . .
0 కూడా (0,0):
0 . .
. . .
. . .
→ X ఎత్తు తుడిచిపెట్టుకుపోయింది.
- గెలిచాక కూడా ఆట కొనసాగుతుంది
X గెలిచాడు
గెలిచాక 0 (2,2) ఆడితే : అనుమతించింది x
ఇప్పుడు board:
X X X
0 0 .
. . 0
- సమం (draw) – ఎవరూ గెలవకుండా board నిండితే
(0,0) → null
... (తొమ్మిదే null)
board నిండింది. ఆట ఏమి చెప్పింది? – ఏమీ లేదు x
X 0 X

X O O
O X X

మూడో విరుపు - ఆటకీ "ఆట" అనేది తెలియదు

1 • ఎత్తు తుడిచేయడం. O గడి (0,0) మీద ఆడింది, మరియు X యొక్క రాయి **మాయమైంది**. Board lo ఇప్పుడు తొమ్మిది గడులు ఉన్నాయి కానీ ఒక్క ఎత్తే.

2 • ముగిసిన ఆట ముగియలేదు. X గెలిచాక కూడా O ఆడగలదు. గెలుపు ఒక సందేశం మాత్రమే అయింది, ఒక స్థితి కాదు.

3 • సమం అనేది లేదు. Board పూర్తిగా నిండింది, ఎవరూ గెలవలేదు, మరియు ఆట **తొమ్మిదిసార్లు null** చెప్పింది. UI ఎప్పటికీ ఎదురుచూస్తూనే ఉంటుంది.

మాలిక తప్పు: `play()` ఒక చర్య మాత్రమే అయింది; అది ఒక **పరివర్తన** కాలేదు. ఒక పరివర్తనకి ముందు షరతులు (ఎవరి వంతు, గడి ఖాళీయేనా, ఆట నడుస్తోందా) మరియు ఒక ఫలితం (గెలుపు, సమం, లేదా తర్వాతి వంతు) ఉంటాయి.

Deep Dive 11 §10 lo ఇదే ఆకారం చూశాం – order ఒకేసారి cancelled మరియు delivered కాగలిగింది, ఎందుకంటే దశల **మధ్య నియమాలు** లేవు. ఇక్కడ ఆట ఒకేసారి ముగిసినది మరియు నడుస్తున్నది.

11. Step – Game ఒక స్థితి యంత్రం

```
class Game {
  #history = [];
  constructor({ rows, cols, players = ['X', 'O'], win, moveRule }) {
    this.board = new Board(rows, cols);
    this.players = players;
    this.win = win;
    this.moveRule = moveRule;
    this.turnIndex = 0;
    this.result = null; // null • {winner} • {draw:true}
  }
  get turn() { return this.players[this.turnIndex]; }
  get isOver() { return this.result !== null; }

  play(move, by = this.turn) {
    if (this.isOver) return { ok: false, reason: 'GAME_OVER', result: this.result };
    if (by !== this.turn)
      return { ok: false, reason: `NOT_YOUR_TURN: ${this.turn}` };

    const spot = this.moveRule.resolve(this.board, move);
    if (!spot.ok) return spot;

    const { r, c } = spot;
    this.board.set(r, c, by);
    this.#history.push({ r, c, by });

    if (this.win.wins(this.board, r, c, by)) {
      this.result = { winner: by, at: { r, c }, moves: this.#history.length };
    }
  }
}
```

```

        return { ok: true, r, c, result: this.result };
    }
    if (this.board.isFull) {
        this.result = { draw: true, moves: this.#history.length };
        return { ok: true, r, c, result: this.result };
    }
    this.turnIndex = (this.turnIndex + 1) % this.players.length;
    return { ok: true, r, c, next: this.turn };
}
}

```

నాలుగు వివరాలు:

- **result** ఒక వస్తువు, ఒక boolean కాదు. `null` = నడుస్తోంది, `{winner}` = గెలుపు, `{draw:true}` = సమం. **మూడు స్థితులు, మూడు రూపాలు.**
- **ప్రతి తనిఖీ ఒక ముందు షరతు** – మరియు అవి **క్రమంలో** ఉన్నాయి: ఆట ముగిసిందా → నా వంతా → ఎత్తు చెల్లుతుందా.
- **గెలుపు తర్వాత వంతు మారదు.** అది సూక్ష్మమైనది కానీ సరైనది – ఆట ముగిసింది, తర్వాతి వంతు అనేది లేదు.
- **players** ఒక array, రెండు స్థిర విలువలు కాదు. ముగ్గురితో ఆడాలంటే `['X', 'O', 'Z']` – code lo ఒక్క మార్పు లేదు.

Part 5 – ఒక engine, మూడు ఆటలు

12. Step – ఎత్తు నియమాన్ని బయటికి తీయడం • గురుత్వాకర్షణ

play lo `this.moveRule.resolve(...)` గమనించారా? – ఒక ఎత్తు ఎక్కడ పడుతుందో అనేది Game కి తెలియదు.

Tic-tac-toe lo నువ్వు చెప్పిన గడిలోనే పడుతుంది:

```
class PlaceAnywhere { // Tic-tac-toe, Gomoku
  resolve(board, move) {
    const { r, c } = move;
    if (!board.inside(r, c)) return { ok: false, reason: `OFF_BOARD: ${r},${c}` };
    if (board.at(r, c) !== null)
      return { ok: false, reason: `OCCUPIED: ${r},${c}` };
    return { ok: true, r, c };
  }
}
```

Connect-4 lo నువ్వు నిలువు వరుస మాత్రమే చెప్తావు; రాయి కిందికి పడుతుంది:

```
class DropInColumn { // Connect-4 – గురుత్వాకర్షణ
  resolve(board, move) {
    const { c } = move;
    if (c < 0 || c >= board.cols)
      return { ok: false, reason: `OFF_BOARD: col ${c}` };
    for (let r = board.rows - 1; r >= 0; r--) // కింది నుంచి పైకి
      if (board.at(r, c) === null) return { ok: true, r, c };
    return { ok: false, reason: `COLUMN_FULL: ${c}` };
  }
}
```

ఇప్పుడు మూడు ఆటలూ మూడు పంక్తులు

```
const ticTacToe = () => new Game({ rows: 3, cols: 3, win: new KInARow(3), moveRule:  
new PlaceAnywhere() });  
const gomoku = () => new Game({ rows: 15, cols: 15, win: new KInARow(5), moveRule:  
new PlaceAnywhere() });  
const connect4 = () => new Game({ rows: 6, cols: 7, win: new KInARow(4), moveRule:  
new DropInColumn() });
```

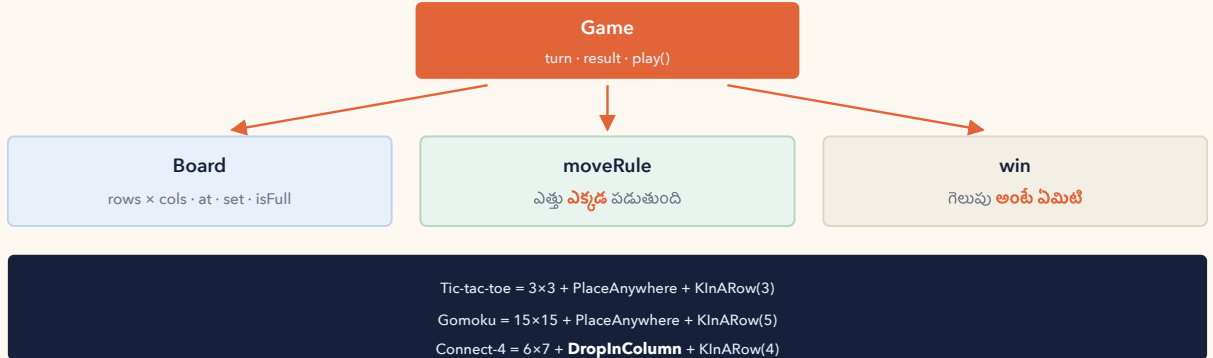
Game class lo ఒక్క if కూడా లేదు – "ఇది Connect-4 అయితే..." అని ఎక్కడా రాయలేదు. ఆట రకం అనేది **ఒక configuration**, ఒక శాఖ కాదు.

మరియు ఇక్కడ **రెండు వేర్వేరు విషయాలని** వేరు చేశాం: ఎత్తు ఎక్కడ పడుతుంది (**moveRule**) మరియు గెలుపు అంటే ఏమిటి (**win**). వాటిని కలిపితే 3×3 దాటి ఏమీ చేయలేం.

13. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

నిర్మాణం · **GAME** కి ఆట ఏదో తెలియదు

మూడు ముక్కలు – ఏ ఆట అనేది వాటిని కలిపే విధానం



```
'use strict';  
const DIRS = [[0,1],[1,0],[1,1],[1,-1]]; // అడ్డం · నిలువు · రెండు వికర్ణాలు  
  
class Board {  
  constructor(rows, cols) {  
    Object.assign(this, { rows, cols, filled: 0 });  
    this.grid = Array.from({ length: rows }, () => Array(cols).fill(null));  
  }  
  inside(r, c) { return r >= 0 && r < this.rows && c >= 0 && c < this.cols; }  
  at(r, c) { return this.inside(r, c) ? this.grid[r][c] : undefined; }  
  set(r, c, v) { if (this.grid[r][c] === null) this.filled++; this.grid[r][c] = v; }  
  get isFull() { return this.filled === this.rows * this.cols; }  
  render() {  
    return this.grid.map(row => row.map(x => x ?? '.').join(' ')).join('\n');  
  }  
}  
  
class KInARow {  
  constructor(k) { this.k = k; }
```

```

// చివరి ఎత్తు (r,c) గుండా మాత్రమే చూస్తాం - అదే మారిన ఏకైక విషయం
wins(board, r, c, p) {
  for (const [dr, dc] of DIRS) {
    let run = 1;
    for (const sign of [1, -1]) {
      let rr = r + dr*sign, cc = c + dc*sign;
      while (board.at(rr, cc) === p) {
        if (++run >= this.k) return true;
        rr += dr*sign; cc += dc*sign;
      }
    }
    if (run >= this.k) return true;
  }
  return false;
}

class PlaceAnywhere { // Tic-tac-toe, Gomoku
  resolve(board, move) {
    const { r, c } = move;
    if (!board.inside(r, c)) return { ok: false, reason: `OFF_BOARD: ${r},${c}` };
    if (board.at(r, c) !== null)
      return { ok: false, reason: `OCCUPIED: ${r},${c}` };
    return { ok: true, r, c };
  }
}

class DropInColumn { // Connect-4 - గురుత్వాకర్షణ
  resolve(board, move) {
    const { c } = move;
    if (c < 0 || c >= board.cols)
      return { ok: false, reason: `OFF_BOARD: col ${c}` };
    for (let r = board.rows - 1; r >= 0; r--) // కింది నుంచి పైకి
      if (board.at(r, c) === null) return { ok: true, r, c };
    return { ok: false, reason: `COLUMN_FULL: ${c}` };
  }
}

class Game {
  #history = [];
  constructor({ rows, cols, players = ['X', 'O'], win, moveRule }) {
    this.board = new Board(rows, cols);
    this.players = players;
    this.win = win;
    this.moveRule = moveRule;
    this.turnIndex = 0;
    this.result = null; // null · {winner} · {draw:true}
  }
  get turn() { return this.players[this.turnIndex]; }
  get isOver() { return this.result !== null; }

  play(move, by = this.turn) {
    if (this.isOver)
      return { ok: false, reason: 'GAME_OVER', result: this.result };
    if (by !== this.turn)
      return { ok: false, reason: `NOT_YOUR_TURN: ${this.turn}` };

    const spot = this.moveRule.resolve(this.board, move);
    if (!spot.ok) return spot;
  }
}

```

```

const { r, c } = spot;
this.board.set(r, c, by);
this.#history.push({ r, c, by });

if (this.win.wins(this.board, r, c, by)) {
  this.result = { winner: by, at: { r, c }, moves: this.#history.length };
  return { ok: true, r, c, result: this.result };
}
if (this.board.isFull) {
  this.result = { draw: true, moves: this.#history.length };
  return { ok: true, r, c, result: this.result };
}
this.turnIndex = (this.turnIndex + 1) % this.players.length;
return { ok: true, r, c, next: this.turn };
}
get history() { return [...this.#history]; }
}

const ticTacToe = () => new Game({ rows: 3, cols: 3,
  win: new KInARow(3), moveRule: new PlaceAnywhere() });
const gomoku = () => new Game({ rows: 15, cols: 15,
  win: new KInARow(5), moveRule: new PlaceAnywhere() });
const connect4 = () => new Game({ rows: 6, cols: 7,
  win: new KInARow(4), moveRule: new DropInColumn() });

```

```

--- Tic-tac-toe (3x3, k=3) ---
(0,0) → 0
(1,1) → X
(0,1) → 0
(2,2) → X
(0,2) {"winner":"X","at":{"r":0,"c":2},"moves":5}
X X X
. 0 .
. . 0

--- తిరస్కరణలు ---
అదే గడి       : { ok: false, reason: 'OCCUPIED: 0,0' }
board బయట     : { ok: false, reason: 'OFF_BOARD: 9,9' }
తప్పు వంతు     : { ok: false, reason: 'NOT_YOUR_TURN: 0' }
ఆట ముగిశాక    : { ok: false, reason: 'GAME_OVER', result: { winner: 'X', at: { r:
0, c: 2 }, moves: 5 } }

--- సమం (draw) ---
{ draw: true, moves: 9 }
X 0 X
X 0 0
0 X X

--- Gomoku (15x15, k=5) - అదే code ---
X (7,3) → ...
X (7,4) → ...
X (7,5) → ...
X (7,6) → ...
X (7,7) → {"winner":"X","at":{"r":7,"c":7},"moves":9}

--- Connect-4 (6x7, k=4, గురుత్వాకర్షణ) - అదే code ---
column 3 → row 5

```

```

column 4 → row 5
column 3 → row 4
column 4 → row 4
column 3 → row 3
column 4 → row 3
column 3 → row 2  {"winner":"X","at":{"r":2,"c":3},"moves":7}
. . . . .
. . . . .
. . . X . . .
. . . X 0 . .
. . . X 0 . .
. . . X 0 . .

--- చెదురుమదురు X లు Gomoku lo (§7 విరుపు) ---
X (7,0) → కాదు ✓
X (7,2) → కాదు ✓
X (7,4) → కాదు ✓
X (7,6) → కాదు ✓
X (7,8) → కాదు ✓

```

మరియు fuzz test

600 ఆటలు · 25,791 చెల్లుబాటుయ్యే ఎత్తులు · 4 ఆట రకాలు
engine = brute force, మరియు ప్రతి ఆటా ముగిసిందా? అవును ✓

రెండు విషయాలు పరీక్షించాం: (1) engine యొక్క గెలుపు తీర్పు **brute-force scan** తో సరిపోతుందా, మరియు (2) **ప్రతి ఆటా ఒక ఫలితంతో ముగుస్తుందా** – §10 యొక్క "సమం అనేది లేదు" సమస్య తిరిగి రాకుండా.

ఈ output ని పంక్తి పంక్తిగా చదువుదాం

(0,0) → 0 – ఎత్తు విజయవంతం, **తర్వాతి వంతు** ఎవరిదో చెప్తోంది. UI కి అదే కావాలి.

నాలుగు తిరస్కరణలూ వేర్వేరు కారణాలతో – **OCCUPIED** , **OFF_BOARD** , **NOT_YOUR_TURN** , **GAME_OVER** .
§10 lo నాలుగూ **నిశ్శబ్దంగా అనుమతించబడ్డాయి**.

{ draw: true, moves: 9 } – board నిండింది, ఎవరూ గెలవలేదు, మరియు ఆట **అలా చెప్పింది**. §10 lo ఇది తొమ్మిది **null** లు.

Connect-4 lo column 3 → row 5 – నేను నిలువు వరుస 3 చెప్పాను; engine **row 5** అని జవాబిచ్చింది. రాయి ఎక్కడ పడిందో అదే. ఆపై 4, 3, 2 – పైకి పేరుకుంటూ, నాలుగోది గెలిపించింది.

X (7,8) → కాదు ✓ – §7 యొక్క తప్పుడు గెలుపు ఇప్పుడు లేదు.

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§3	Board + checkWin	మౌలిక అస్థిపంజరం
§4 (విరుపు)	రేఖల గణనలు	ఒక ఎత్తుకి 1,80,600 తనిఖీలు
§7 (విరుపు)	KInARow – దిశల వరుస	గణనలు $K < N$ కి తప్పు జవాబులు

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§8	Fuzz vs brute force	సరిహద్దులు చాలా, tests చాలవు
§10 (విరుపు)	<code>result</code> , ముందు షరతులు	ఎత్తు తుడిచేయడం, ముగియని ఆట
§11	<code>players</code> ఒక array	ముగ్గురితో ఆడటం ఉచితం
§12	<code>moveRule</code> విడిగా	Connect-4 lo రాయి కిందికి పడుతుంది

Part 6 – Interview Io

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

45 నిమిషాల TIME BUDGET

3×3 ని వేగంగా చాటుండి – నిజమైన సమయం సాధారణీకరణ

5 నిమి

Clarify – N×N? K వరుసగా? గురుత్వాకర్షణ?

7 నిమి

3×3 · scan ఎందుకు సరిపోదు

13 నిమి – KInARow

counters · అవి ఎక్కడ విరుగుతాయో · run check

11 నిమి – Game

result · ముందు షరతులు · draw

9 నిమి

moveRule · Connect-4 · AI · undo

ఏమి తప్పక చెప్పాలి

1. "ఈ ఎత్తు గెలిపించిందా?" (§4) – "ఎవరైనా గెలిచారా?" కాదు. ఇదే మొత్తం ఆలోచన.
2. Counter ఉపాయం, మరియు అది ఎక్కడ విరుగుతుందో (§7) – ముందు అది చెప్పి, ఆపై దాని హద్దు చెప్పండి. `X.X.X.X.X` ఉదాహరణ గీయండి.
3. Run check $O(k)$, board పరిమాణంతో సంబంధం లేదు (§8) – మరియు అది $K = N$ కి కూడా పనిచేస్తుంది.
4. result ఒక వస్తువు; draw ఒక ఫలితం (§11) – చాలా మంది draw ని పూర్తిగా మర్చిపోతారు.
5. moveRule విడిగా (§12) – "Connect-4 is the same engine with a different move rule." ఇది ఒక వాక్యం, మరియు అది interview ని ముగిస్తుంది.

ఏమి వదిలేయాలి

- Counter class ని పూర్తిగా రాయొద్దు – నాలుగు పంక్తులు చెప్పి, హద్దు చెప్పి, run check కి వెళ్ళండి.
- bruteWins రాయొద్దు – "I'd fuzz against a brute-force scan" అని చెప్తే చాలు.
- AI (minimax) – అడిగితేనే (§16).
- UI, rendering – `board.render()` ఒక పంక్తి, అంతే.

15. నీటితో చెప్పాల్సిన English script

"Three questions first. Is the board fixed at three by three or N by N? Does winning need a full line, or K in a row? And is there gravity, like Connect-4?

I ask the second one because it changes the algorithm completely, and it's easy to miss since they're the same thing on a three by three board.

The naive version checks every row, column and diagonal after each move. That's fine at three by three and quadratic per move as the board grows – I measured it, and a three-hundred by three-hundred game did about a hundred and eighty thousand cell checks per move and took thirty-five seconds. The tell is that the player changed one cell and we re-examined the whole board.

If winning means a full line, there's a nice trick: keep a running count per row, per column and per diagonal, plus one for the anti-diagonal. X adds one, O subtracts one. A line is won when the absolute value hits N, because mixed marks cancel. That's four operations per move regardless of board size, and it's the standard answer to LeetCode 348.

But I'd flag straight away where it breaks. It quietly assumes K equals N. For Gomoku – fifteen by fifteen, five in a row – five consecutive X's give a count of five, and the check is looking for fifteen, so it misses the win entirely. And the obvious fix, comparing against K instead, is worse: five X's scattered across a row with gaps also sums to five, so it declares a win where no two stones are adjacent. The count tells you how many, not where.

So I'd use directional run counting instead. After placing a stone, walk outward from it in four directions – horizontal, vertical, and both diagonals – counting matching stones on both sides. If any run reaches K, that's a win. It's $O(K)$ per move rather than $O(1)$, but K is small, it's independent of board size, and it's correct for K equals N too, so one implementation covers every variant. I measured about nine steps per move at every board size, versus nearly two seconds of brute scanning at three hundred by three hundred. And I'd fuzz it against a brute-force scan, because the edge cases – board borders, runs that span both sides of the new stone – are easy to get subtly wrong.

The part people skip is the game state itself. A play method that just writes to the board and checks for a win lets you play on an occupied cell and silently erase someone's move, lets play continue after a win, and never detects a draw – a full board with no winner just returns null nine times and the UI waits forever. So play is a transition with preconditions: is the game over, is it your turn, is the move legal – and it produces a result: a winner, a draw, or whose turn is next.

Last, I'd pull out where a move lands as its own rule. Tic-tac-toe places where you point; Connect-4 drops to the lowest empty row in the column. With that separated, Connect-4 is the same engine with a different move rule and K of four – no conditionals inside the game class at all."

16. Follow-ups – AI, undo, ఆన్‌లైన్ ఆట

Follow-up	జవాబు	మారే classes
"ముగ్గురు ఆటగాళ్ళు"	<code>players: ['X','O','Z']</code>	0
"ద్వీపచతురస్ర board (6×7)"	<code>Board</code> ఇప్పటికే <code>rows × cols</code>	0
"Undo"	<code>#history</code> ఇప్పటికే ఉంది – చివరి ఎత్తు తీసి, <code>result</code> ని <code>null</code> చేయడం	+1 method
"ఆట స్థితిని save/load"	<code>#history</code> ని serialize చేసి, మళ్ళీ ఆడించడం	+2 methods
"ఎవరు గెలుస్తారో ముందే చెప్పగలమా (3×3)?"	3×3 ఆట పరిష్కరించబడినది – సరిగ్గా ఆడితే ఎప్పుడూ draw	–
"AI ఎదురుగా ఆడాలి"	కింద చూడండి	కొత్త layer
"ఆన్‌లైన్ – ఇద్దరు వేర్వేరు చోట్ల"	కింద చూడండి	Server layer

AI – ఒక కొత్త layer, ఒక కొత్త concept కాదు

"3×3 కి **minimax** సరిపోతుంది – ఆటల చెట్టు చిన్నది (9! కంటే తక్కువ). ముఖ్యమైన విషయం: minimax కి **ఒక Game నకలు** కావాలి, ఎందుకంటే అది ఎత్తులు వేసి చూసి వెనక్కి తీయాలి.

నా `#history` ఉంది కాబట్టి **undo** సులభం – అదే minimax కి కావాల్సినది. కాబట్టి ఇది ఒక కొత్త layer, కొత్త design కాదు.

Gomoku కి minimax పనిచేయదు – 15×15 lo శాఖల సంఖ్య చాలా ఎక్కువ. అక్కడ **alpha-beta pruning + ఒక heuristic** (బహిరంగ మూడు, బహిరంగ నాలుగు వంటి నమూనాలకి score) అవసరం. ఆ తేడాని చెప్పడం – మీరు సమస్య పరిమాణాన్ని గుర్తిస్తున్నారని చూపిస్తుంది."

ఆన్‌లైన్ ఆట

"ఇద్దరు వేర్వేరు చోట్ల ఆడితే, **నిజం ఎక్కడ ఉంది** అనేది ప్రశ్న. జవాబు: **server lo**. Client ఒక ఎత్తు పంపుతుంది, server `play()` పిలుస్తుంది, ఫలితాన్ని ఇద్దరికీ పంపుతుంది.

నా `play()` ఇప్పటికే దీనికి సిద్ధంగా ఉంది – ఎందుకంటే అది `by` అనే parameter తీసుకుంటుంది మరియు `NOT_YOUR_TURN` తిరస్కరిస్తుంది. Client ని నమ్మాల్సిన అవసరం లేదు.

మరియు నవీకరణలు పంపడానికి – Deep Dive 07 యొక్క event bus, లేదా WebSocket. **0 కొత్త concepts.**"

17. ఏమి నేర్చుకున్నాం

అలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
"ఏమి మారింది?" అని అడగడం	చివరి ఎత్తు గుండా మాత్రమే (§8)	Deep Dive 12 §5, 13 §4, 14 §11
వేగవంతమైన ఉపాయాల దాచిన ఊహలు	Counters $K = N$ అనుకున్నాయి (§7)	ప్రతి "తెలివైన" optimisation
తప్పు జవాబు error కంటే ప్రమాదకరం	X.X.X.X.X గెలిచాడు (§7)	Deep Dive 11 §8, 13 §5
చర్య కాదు – పరివర్తన	play() ముందు షరతులు (§11)	Deep Dive 11 §11 · ప్రతి state machine
మారేదాన్ని బయట పెట్టు	moveRule , win (§12)	Deep Dive 01, 03, 05, 06, 10
ముగింపు ఒక ఫలితం	{draw:true} (§11)	Jobs, transactions, workflows
Fuzz vs brute force	32,287 + 25,791 ఎత్తులు (§8, §13)	Deep Dive 14 · ఏ optimisation ఐనా

ఒక చివరి అలోచన – ఈ PROBLEM ఎందుకు మోసపూరితమైనది

Tic-tac-toe ని చాలా మంది "వేడెక్కించే ప్రశ్న" అనుకుంటారు, మరియు అదే ప్రమాదం.

3×3 board మీద ప్రతి పద్ధతీ పనిచేస్తుంది. కాబట్టి interviewer మీ మొదటి పరిష్కారాన్ని చూసి ఏమీ తెలుసుకోలేడు. అతను తెలుసుకునేది – మీరు దాన్ని ఎంత దూరం తీసుకెళ్ళగలరే.

మరియు ఈ doc lo చూసినట్లు, ఆ దారిలో ప్రతి అడుగులోనూ ఒక నిశ్శబ్ద తప్పు ఉంది: counters $K < N$ కి తప్పు, threshold సవరణ మరింత తప్పు, మరియు win check మీద కష్టపడుతూ ఆట నియమాలనే మర్చిపోవడం.

"సులభమైన" problem అంటే – తప్పులు కనిపించని problem.

ఇక్కడి నుంచి ఎక్కడికి

ఈ series lo ఇప్పటివరకు: **01 Parking Lot · 02 Cache · 03 Rate Limiter · 04 BookMyShow · 05 Splitwise · 06 Elevator · 07 Pub-Sub · 08 HashMap · 09 Chess · 10 Meeting Scheduler · 11 Food Delivery · 12 File System · 13 Leaderboard · 14 Text Editor · 15 Library · 16 Tic-Tac-Toe.**

Deep Dive 09 (Chess) దీని పెద్ద సోదరుడు – అక్కడ ఎత్తు నియమాలు ముక్కని బట్టి మారతాయి, మరియు "చెల్లుబాటుయ్యే ఎత్తు" అనేది మొత్తం board మీద ఆధారపడుతుంది.

Tic-Tac-Toe → Board Game Engine – అడుగు అడుగునా · ఈ doc lo ఉన్న ప్రతి output నిజంగా node lo run చేసి తీసినదే ✓